

Ben Meyers
October, 2025

Why Overparameterization Works

Double Descent Justified with Energy Normalization

Abstract

In this composition, we explore the phenomenon of “Double Descent”, the tendency of highly overparameterized machine learning models to interpolate their testing data, instead of overfitting it, as conventional wisdom warns it will. In recent years, deep machine learning has seen that massive neural networks somehow skirt overfitting and powerfully generalize from their training. There have been several key findings and advancements in the treatment of deep neural networks that have made *such deep* learning successful at its scale—such as better activation functions, optimal weight-initialization techniques, intra-layer normalization, and others—but here we primarily discuss the role of L_2 normalization. With normalization, overly complex models can be incentivized towards *lower energy* interpolations of their training data, getting the benefit of a highly expressive function space (endowed by the overly complex model), but minimizing chaotic deviation (achieved by the L_2 punishment). Importantly, double descent has been observed even without explicit L_2 regularization—arising from implicit biases in gradient descent toward minimum-norm solutions. Our focus here is on using energy minimization as a theoretical lens to explain *why* these low-complexity interpolations generalize better, whether that minimization occurs explicitly (via L_2 normalization) or implicitly (via optimization dynamics). Finally, make an analogy between low energy interpolations and Minimum Description Length programs, highlighting the common approach toward elegant and efficient characterization of a function.

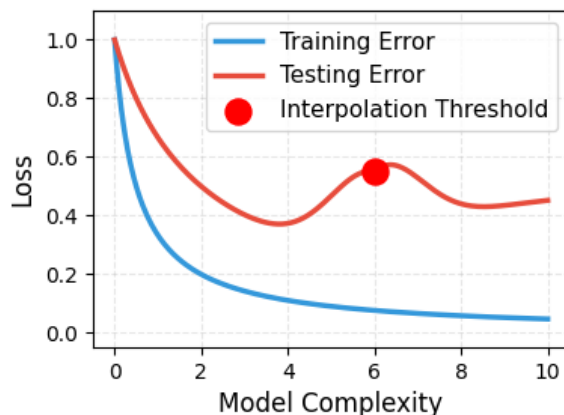
Overfitting

In machine learning, the conventional wisdom cautions against overfitting. Overfitting is a phenomenon where the statistical model you train on a dataset (which you hope embodies a fair sample of some true distribution which you’d like the model to *learn*) picks up on noise in the data, not generalizing insight from it but instead just memorizing it and all of its irrelevant idiosyncrasies. An overfit ChatGPT would not be able to meaningfully say anything deviant from its training data, would not seem to have actually generalized linguistic patterns, and would simply not be useful. An overfit face identification software would not permit entry to any but the set of images you used to calibrate the tool. When we train programs based on statistics, we want generalization from the training data so the model can apply its understanding to new scenarios.

To mitigate overfitting (or at least measure it), engineers typically partition the data so most of it is used for training, but a percentage is saved for out-of-sample validation. If the model does not train on this *test* data, you know it could not have been memorized. Further—to spell this out completely—if the model does well out-of-sample, you get the clue that it is generalizing successfully.

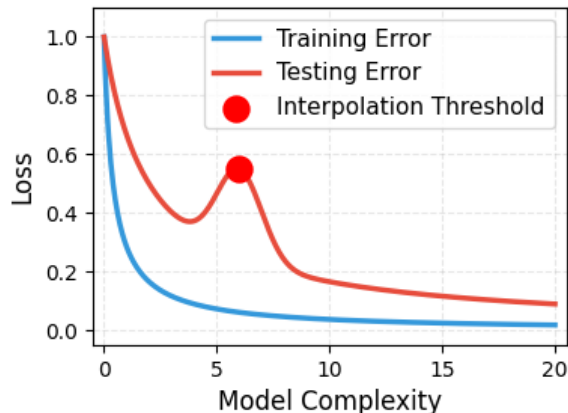
When you plot the training error and the testing error against the complexity or size of your model (or steps of training¹), you typically see that both errors descend for a period, and then at a certain point (called the interpolation threshold), the *testing error* peaks. It is typically at this point that models can be said to have overfit their data. It is the point when the model’s complexity matches exactly that of its training dataset, allowing for full memorization.

¹ For a given model of a given size, it is also typical to see plots like this, where testing error rises after a certain point, indicating that the model is overfit. However for the phenomenon we seek to describe, we care about how the testing error (after some fixed amount of training for each model complexity) changes with complexity. Just as models that train *too long* can pick up noise, those with too many degrees of freedom are more susceptible to this overfitting.



Assuming that train and test data represent the same domain, seeing a divergent testing error suggests that idiosyncratic noise (signal that is unrelated or incidental to the domain you really care about) has taken root in the model. This is the pattern that always tends to crop up when you scale up models. So we typically stop scaling at that interpolation threshold! With too much freedom endowed to the model in order for it to be shaped by the contours of the data, it can be impressed more easily by noise.

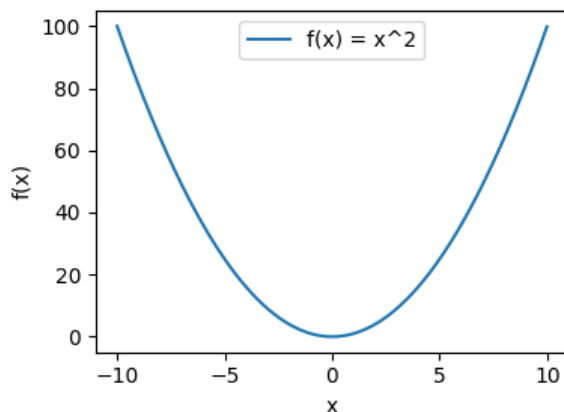
In recent years, surprisingly, as models have grown in complexity and training regimes have scaled up, this divergent testing error indicative of overfitting has been seen to collapse back down with the training error!



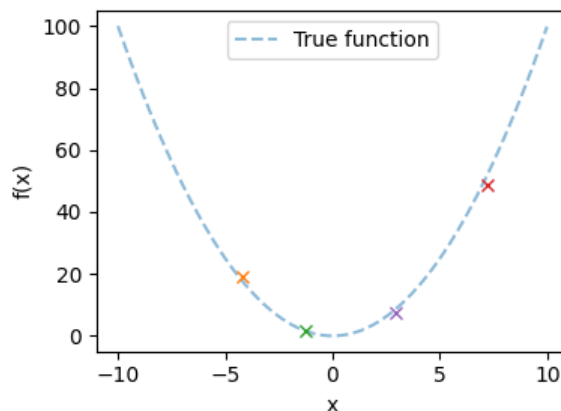
As we keep adding complexity to the model, the testing error crashes back down and we see signs of real generalization in the model. So what could be causing this *double descent*, as it has been deemed? How could the testing error, after departing duly from the training error, crash back down, indicating a very well-generalized model?

A Simpler Example

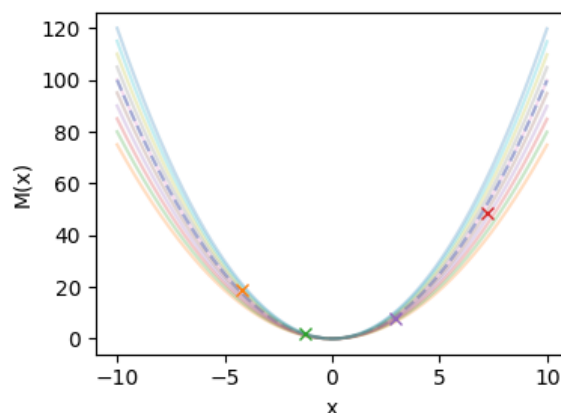
While the phenomena we discuss apply emphatically to deep neural networks, we can highlight the fine points of this double descent with a far lower-order example. To accentuate the effect, we will use the function $f(x) = x^2$ as the target for our model to learn and we will only offer 4 training examples.



In truth, this function can be perfectly represented (indeed it is defined) with the following solution polynomial: $M(x) = 0 + 0x + 1x^2$; that is, only three parameters (0,0,1) are needed to be learned by our function to achieve our goal. (And it actually looks more like one.) Trying to add an x^3 or x^4 term would add nothing to the accuracy of our model. However while we can clearly see the solution (we literally define the answer for our model to reverse engineer!) for this specific, low-dimensional example, when an optimization algorithm is set blind against a dataset, often the function is far more complex. Here we show our ‘true’ curve with a dotted line and our training points marked by X-es:



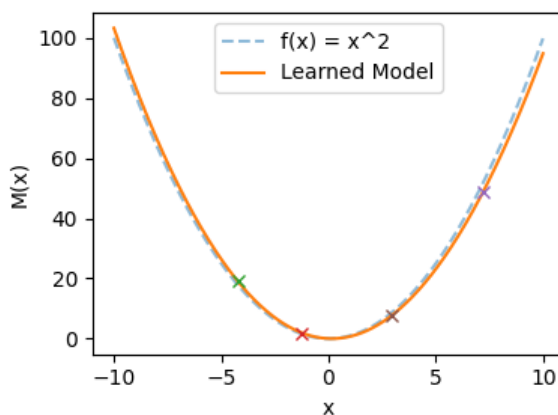
All that our model will see are these points and we seek to minimize its prediction error. An n -degree polynomial, in general, consists of $n+1$ parameters. If we set up the architecture of our model as a second-degree polynomial, this means we will let it learn the three parameters, P , in the following function: $M(x) = p_0 + p_1x + p_2x^2$. We aim to learn: $P = [p_0, p_1, p_2]$. If we use gradient descent, then we will easily and unsurprisingly converge to: $M(x) = 0 + 0x + 1x^2$, or $P = [0, 0, 1]$ and achieve 0 error. Any function in a local region (here pictured is $p_2 \in [0.75, 1.25]$) will fit our data very well:



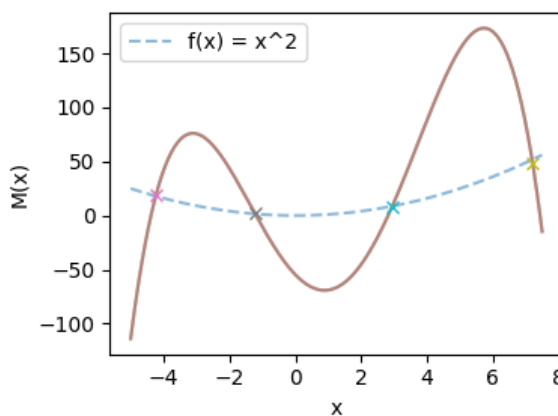
Being of the order of x^2 , a model with parameter dimension 3, this architecture has everything it needs (we know, again, because this example is contrived) to succeed. What happens, however, if we add more dimensions? Two aspects we should keep in mind are (a) the size of the parameter set, P , versus the parameter size of the true function, which is three, and (b) the size of the parameter set, P , versus the number of training examples we have. This is a vital distinction because it reminds us what explicit and implicit jobs machine learning models have.

Implicitly, we train them to learn an underlying function (here $f(x) = x^2$). While it seems (is) tractable in our example, in real contexts, these functions are unknown and complex (if they were known, machine learning them would be obviated!). In practice, though, a rough idea for at least the band of dimensionality that a problem demands is good to keep in mind.

Explicitly, ML models are trained to fit data points. The sole objective (in a straightforward model and error function) is to minimize the distance between the model's predicted answers and those in the training dataset. Our explicit goals seek to envelop the implicit ones, but this is not at all guaranteed. In our training set, there are only 4 points. If we have 4 parameters in our model (if we add one term, p_3x^3) then we can also perfectly fit our data. This would be a *determined* system of equations, because the freedom of the model envelops that of the data. There are several methods to achieve this determined solution, but the intuition is that with as many degrees of freedom as there are free points to learn, then we expect that the model will not have to stretch itself at all to attend to each. If we solve such a perfectly-determined system of equations, we can get: $M(x) = 0.034 - 0.297x + 0.99x^2 - 0.0011x^3$, or $P = [0.034, -0.297, 0.99, -0.0011]$. Below are the results:



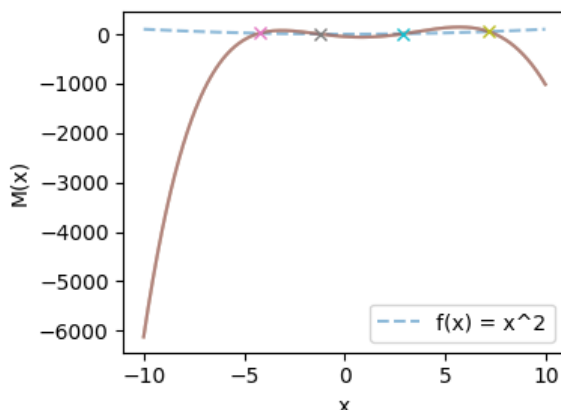
If we increase the size of our model past this point, we will, on both fronts, have too much freedom in our model and will be subject to overfitting. We will not bother discussing the training process, but here is a sample *5-parameter* function that could have been learned to fit these 4 data points from this 3-parameter function:



We have zoomed in around our data points for illustrative effect, but we clearly see some problems. If we look at error alone, this model is perfect. Clearly, though, this approximation is ignorant to the true

function defining our points, but that is a fact only we can see. When you have too many parameters, you can *overfit* the data—by the computed error, these are successful models, but upon examination, we feel that they are somewhat cheating!

The following shows a model with 7 parameters fitting the points perfectly, but this time we zoom out to highlight the insane *coincidence* it seems that the data are predicted:



This is overfitting gone totally unchecked. So earlier we said that, despite the conventional wisdom discouraging the above catastrophe, with many more parameters than are needed, models could still generalize from data. How could this be achieved given what we see? Let's pause for some intuitive thinking...

Intuition Behind a Solution to Overfitting: Energy

While these examples of overfit models clearly hug the data perfectly, they seem to be *doing too much*. Even though they could have learned $P = [0, 0, 1, \dots, 0]$, they have parameter sets which walk the tightrope of the datapoints with far less elegance. What notion are we really getting at when we say *elegance* or *doing too much*? Can we quantify—or at least explicitly describe—all of this extra work? Ahh *work*...

Physics is the study of matter and motion and *energy*. In physics, the concept of *work* denotes the degree (over space, mass, and time) to which energy was exerted in getting something done. If I want to walk to the north side of Central Park from the middle, I will do considerably less work simply walking north than I would if I first go to the south side, turn around, then walk north. The same goes for mass; when I lift my infant baby cousin, I do considerably less work than when I try to lift my 18-year old brother. Work is a measure of energy applied to a situation.

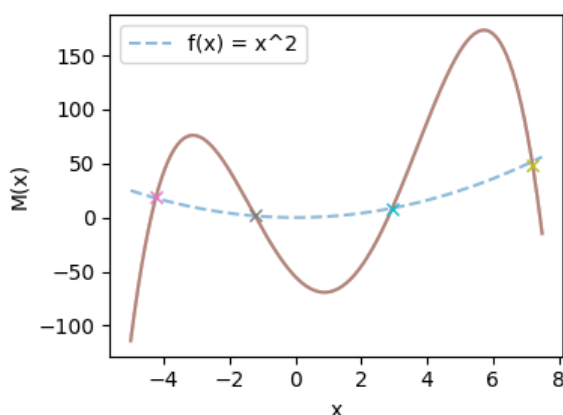
Looking at the solutions to our dataset, we can see work once again. Our *true* solution, $P = [0, 0, 1, \dots, 0]$, is doing the least possible work to achieve its task and the other solutions we've seen— with their extra variables, very weird and specific decimal parameters, and wild curves up and down, and clear ignorance of the pattern. (Ahh, *pattern*, this will be discussed in due time.) Again in our contrived example we see this clearly: we already had the solution in three dimensions, so adding dimensions was already extra work. Any non-zero values, $p_i \in \{\neg(n = 0) \mid N\} \wedge \neg(i = 2)$, get assigned to parameters other than p_2 will have to be balanced out by other parameters in the model. Any deviance from $P = [0, 0, 1, \dots, 0]$ will require a *counterforce*, so to speak, provided by other parameters. The balance of that true solution will be disturbed. Wherever the curve goes up when it shouldn't—like between $x = 3$ and $x = 7$ in our 5-parameter solution's plot—demands other parameters of the model to pull it back down to the next point.

Another way to think about this: the sum of the parameter values in our perfect solution is 1. Summing, though, loses some information; when we see this sum, we have no indication that the full solution is

$P = [0,0,1,\dots,0]$ and not $P = [1,0,0,\dots,0]$ or $P = [0,0.5,0.5,\dots,0]$. This is an important consideration, but at least we can say that if the sum of our parameters is greater or less than 1, then we know we should be suspicious. Ah, you say, but what if the sum is 1 because some terms are positive and some are negative? This is why we measure not just the naive sum, but the sum of the *squared* values of each parameter. This metric will then give us an idea of *magnitude* of our solution. It attributes a kind of *weight* to our parameter set. This squared sum is called the $L2$ score.

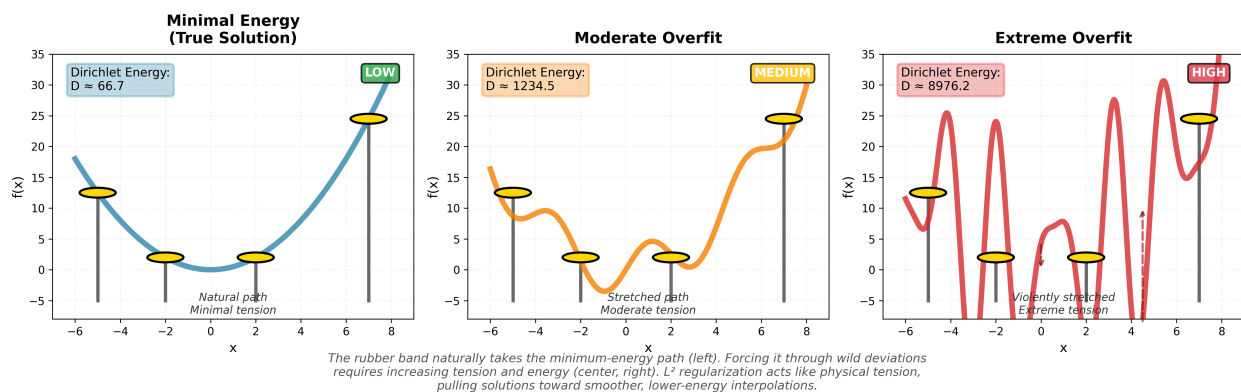
When you have lots of cancelling with positive and negative parameters, you will have a high $L2$ and you can deduce that your model is doing lots of work. This notion of work in Physics applied to this situation still feels somewhat unearned. How about an analogy that will perfectly marry the two situations.

Consider again this plot of our overfit 5-parameter solution:



Imagine that the points are not abstract coordinates in the X-Y plane but instead little rings pegged vertically to the ground or some table. You could also think of them as loops in some kind of loom. Consider then that our *true* dotted line solution is a rubber band that has been cut, its ends pegged down on the table at the first and last datapoint, and it has been run through the loops. This configuration, being the true minimum work solution, has just enough length to make it through this course and the string will not have extreme tension.

The Rubber Band Analogy: Energy and Overfitting



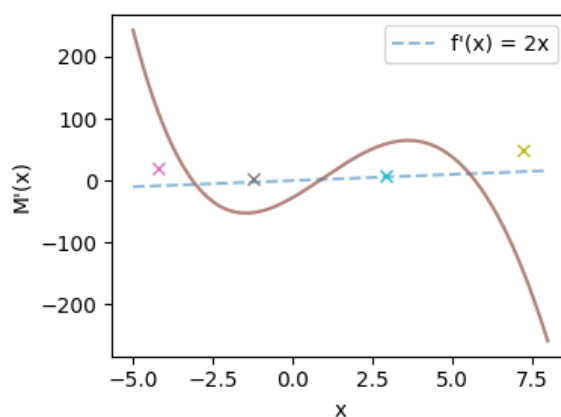
Consider, though, what kind of work would be required to stretch this already-fit band to resemble the solution of our overfit polynomial. To deviate from this minimum distance (and thus minimum tension)

solution would require you to exert force in counteracting the rubber band's pull back to equilibrium. Every degree it is pulled off that short solution will demand extra work.

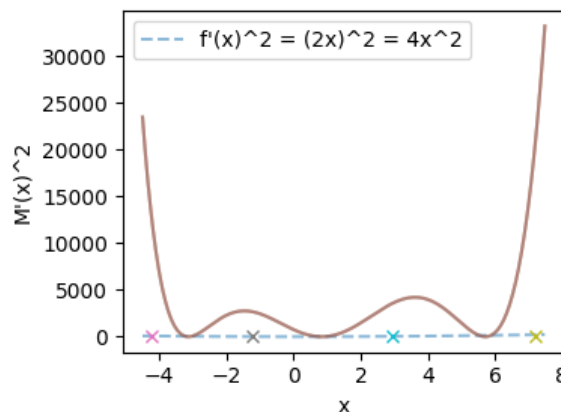
Thus the true solution is the one which requires the least amount of energy. Energy, we now know, can be thought of physically as well as numerically in curve-fitting space. Let's take one step further into the domain of physics to tap its wellspring of notation, solution, and intuition in this question of energy.

To formalize our notion of energy in rubber bands and curve-fitting solutions, we can discuss the Dirichlet energy equation. It is defined as: $D = \int (f'(x))^2$ or $D = \int \left(\frac{df(x)}{dx} \right)^2 dx$. In English, this means the

Integral of the squared Derivative of $f(x)$. In intuitive English, this means the *net magnitude of the forces of change in the function*. The derivative of a $f(x)$ gives a measurement, at each x , of how much $f(x)$ will change in the next instant. The derivatives of both our true function and of our overfit function is shown below (true, the dotted line):



We see that the real function changes at a constant rate (2). The approximated function, though, starts off by changing too positively and then balances itself by changing too negatively. Look what happens when we square these derivatives:

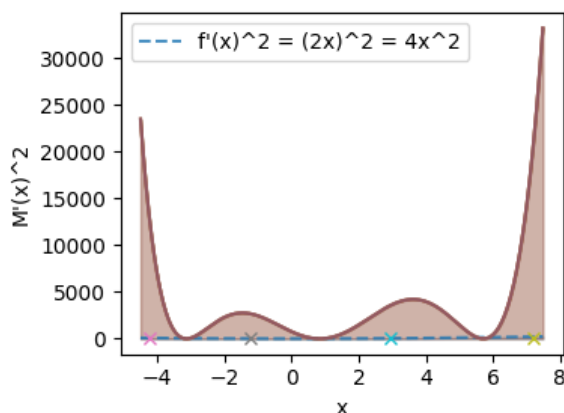


The integral of a function tells us the *area under its curve*, giving a value that grows exponentially with *height*. Since we are taking the integral of *the derivative squared* of our function, the height will represent

the squared degree to which the function changes—i.e. a high value means a function that changes dramatically.

Looking at this derivative squared on the plot, we see immediately that the area under our approximated function's squared derivatives will be massive in comparison to that of the true solution. $f'(x)^2 = 4x^2$ is a quadratic function that actually looks like a stretched version of the original $f(x)$, but you'd never know that when the monstrous squared derivative of the approximation sits behind it, warping the scale of this already-zoomed-in graph.

The energy of these approximated solutions is massive, as shown explicitly below. The energy of the true solution is also plotted as an area, but it cannot be seen!

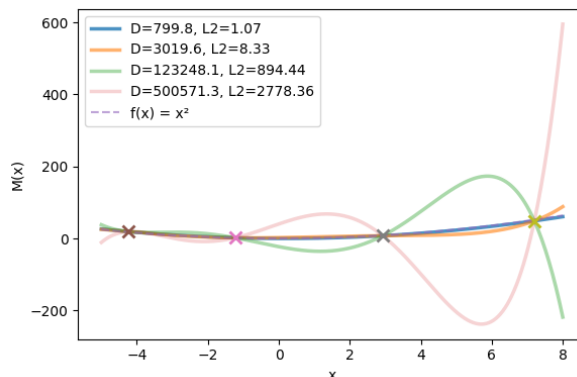


Dirichlet energy and $L2$ score are distinct measures. However, Dirichlet energy is indirectly, analogically proportional to the $L2$ value: $L2 = \sum_i^P p_i^2$. This is the sum of the squares of the parameters, but look what the Dirichlet is computing: $\sum_i^P (ip_i(x)^{i-1})^2$. The latter depends on values of x as well as the degree of the parameter (to what power x is taken in its term), but the functions are still both proportional to p_i^2 . $L2$ loses information contained in the Dirichlet equation, but they both crucially measure the energy of the solution dependent on the magnitude of its parameters. Dirichlet provides an elegant theoretical justification for why $L2$ works.

Energy-Inspired Solution: $L2$ Normalization

Part of the solution to the problem of overfitting in overparameterized models is an energy penalty in the loss function used to optimize the parameters of the model. That is, instead of just incentivizing the data points being fit, but we incentivize *low energy* fits. Without this term, the sole attracting force in the loss function (the guiding value in the training algorithm) is the error in predicting values in the training data. This blindly seeks to fit the rubber band through the loops, with no attention paid to energy or work. When we penalize *energy* (approximated by $L2$) in this loss function, we have an attractor at the lowest energy, lowest error solution.

Even in highly overparameterized models, when we punish for high-energy solutions, we find a solution that actually *interpolates* the training data—meaning it fills in the space between points with the shortest curve segments. Below are several approximations of $f(x) = x^2$ with varying Dirichlet and $L2$ values:



As the curves approach the true solution, their energies decrease. This should really be phrased in reverse! Though each of these solutions have error of 0, as we incentivize lower energy, the solutions interpolate the data and approach the real solution much more elegantly.

This demonstrates a vital secret behind how massively overparameterized models—like modern Large Language Models—do not threaten their generalization with overfitting. Yes, the training data is effectively memorized, however the space between the data points—the curve we aim to interpolate—is properly represented. Thus we see that low energy interpolation *is* generalization.

Supposing you *set* your phone's face recognition or fingerprint scanner with just a handful of examples, the fact that you can get in with new inputs (your face in the camera or finger on the sensor) that are certainly objectively different than these examples is due to the models interpolating the contours of your real face or finger. The fact that LLMs can carry arbitrary conversation that *certainly* did not exist in its training data is because they—being stuffed with parameters but also coaxed towards low energy—have interpolated the curves of syntax, semantics, and fact.

Wait, but LLMs are trained on *trillions* of examples of text...are they really made of *many more than trillions* of parameters?

Effective Complexity

Our example was illustrative of the mathematical effects of overparameterization and $L2$ normalization. However it paints a slightly misleading image. In our model, we had a single polynomial whose degree (number of parameters - 1) represented its *effective* degree and *effective* complexity. But in machine learning models such as Multi-Layer Neural Networks, the effective complexity or degree is not at all just the number of parameters it has.

A neural network is essentially a *series* of polynomial functions like the one we examined, applied iteratively to previous results.² They can be defined as: $NN(x) = f_n(f_{n-1}(\dots(f_0(x))\dots))$. It can be written more imperatively as:

$$\begin{aligned} NN(x) &= f_0(x) \\ NN(x) &= f_{\dots}(NN(x)) \\ NN(x) &= f_{n-1}(NN(x)) \\ NN(x) &= f_n(NN(x)) \end{aligned}$$

² Importantly, they are usually functions of a *vector*, x . That is, they do not contain functions with x terms to any power greater than 1. They instead compute linear combinations of *many* variables. Instead of *high order* functions, they consist of *high dimensional* functions. We will stick with nested polynomials, though, because they illustrate exactly the same notion of how shallower functions (n-degree polynomials or n-dimensional linear combinations) compose into far more complex functions.

Added to this complexity is a nonlinearity that we need not even discuss.³ Just composing these layers on top of each other already expands the expressiveness and complexity of the function multiplicatively. Let's work through an example (again with polynomials, not linear combinations).

Let's look at two models each with the same number of parameters. We have a single 7-degree polynomial: $f(x) = p_0 + p_1x + p_2x^2 + p_3x^3 + p_4x^4 + p_5x^5 + p_6x^6 + p_7x^7$. This has 8 parameters, P .

And then we have a sequence of 3-degree polynomials composed: $g(x) = q_0 + q_1x + q_2x^2 + q_3x^3$ and $h(x) = p_0 + p_1x + p_2x^2 + p_3x^3$ becoming $f(x) = h(g(x)) = p_0 + p_1(g(x)) + p_2(g(x))^2 + p_3(g(x))^3$. This also has 8 parameters, $Q \cup P$.

Even in this last term, we have $g(x)$ (who has a maximum power of 3) taken to the power of 3, giving a maximum degree in this composition of $3 \times 3 = 9$. Even though both functions are cubic, their repeated application *multiplies* these powers, it doesn't just add them. But beyond just a higher final term, we have an explosion of expression because of the composition of these functions.

A term like $g(x)^2$ is expanded to: $q_0^2 + 2q_0q_1x + (q_1^2 + 2q_0q_2) + \dots + q_3^2x^6$. These are complex interactions between the parameters, Q , of $g(x)$, all now being controlled by the parameters, P , of $h(x)$. P controls functions not just of a single order of x , but of already-multi-order functions of x .

We can think of polynomials of degree n as compositions of the pure functions, $x^0 \rightarrow x^n$. These are the *functional pieces* (you could even think about slices of lines and curves) at the disposal of the polynomial to define its transformation. When these pieces, though, are *already warped* functional material, then the compositions they endow get exponentially more complex.

If we think back to our examination of the derivatives at play, we can understand even more intimately why composition explodes the complexity and expressibility of a function. The highest rate of change (derivative) available to the single polynomial was on the order of x^6 . The highest available to our composed function is x^8 . In any given region of the input domain of x , our composed function simply has more extreme tools at its disposal.

You can think about these derivatives as degrees of nuance in the model; the whole point is to represent a complex and nuanced domain in the world as an approximated function. If there are very sensitive regions of the domain, your model will need precise and discerning expressiveness to be able to attend to that sensitivity. For example, the phrases *he read* and *he reads* are very similar inputs objectively, however the slight difference with the addition of the letter **s** has large implications to the entire syntactical structure of the sentence in which it appears. The slope between these points necessarily needs to be higher than between, say, *he drank 99 bottles of beer* and *he drank 100 bottles of beer*.

So in domains this nuanced, you certainly need many degrees of freedom in your model. However you want constructive interpolation—generalizing from *read/reads* to *cut/cuts*, *hurt/hurts*, and all other irregular verbs—and not noisy memorization. Using an overparameterized model could be said to *afford* you all the nuance you need, and normalization discerns and optimizes what potential nuance should be employed. It could be thus argued that under- or duly-parameterized models miss the potential for expressibility that could, if needed, be tamped down by normalization.

Low Energy $\approx$ Short Description

In computational complexity there is an idea of Minimum Description Length (MDL) of a program. Strictly, the MDL of a function or an output is the shortest possible program (written in binary) for a Turing Machine to achieve some result. As Turing Machine programs are composed of the minimal

³ Each of the above intermediate results is sent through a function like *Sigmoid* which maps $(-\infty, \infty) \rightarrow [0,1]$ or *Hyperbolic Tangent* which maps $(-\infty, \infty) \rightarrow [-1,1]$. These add lots of expressive power to these models by allowing them to abstract, say, two different very large results as 'the same value' or really 'the same input'. This is a mechanism to enrich generalization. It also increases the effective complexity of the model, but already the repeated polynomials give us plenty to analyze.

spanning bases of computation, the shortest *Turing* program is the most compressed, most efficient way to characterize that function.

You can thus classify functions or problems by their MDL or Kolmogorov Complexity. The least compressible functions or outputs are *random* functions or outputs. There is no way to compress a random string of symbols to anything but the string itself. However a function/output such as *10101010...* can be perfectly described or encoded by a program which says the effect of: *write 1 then write 0 then write 1 then...* In general, when modeling functions that we do not *know explicitly* (such as grammatically correct English), we want solutions that are maximally efficient and compressed. We do not want a single *parameter* for every possible input scenario! Composition of functions helps, but, as we saw, it can lead to dangerous overfitting. *L2* or energy-based normalization enforces the constraint towards efficiently compressed solutions.

Once again, this makes perfect sense. If my program to produce *10101010...* was written in any way other than *write 1 then write 0 then write 1 then...*, then necessarily all of the superfluous deviations would need to be 'countered' by yet more corrective terms. More space would be needed for storage of the program (more work), more actual steps would be needed in order to achieve the same result (certainly more work).

In machine learning we do not actually expect a minimum-size description of our problems, however with normalization on complex models, we can at least approach them in training.

One Last High Energy Deviation

To add one entirely gratuitous analogy to this notion of efficiently overfitting a curve, I bring up Chopin's Etude Opus 25 no. 11 in A minor. In this piece, there is a primary melody, a main thrust which guides the song and pleases the ears. However around this smooth curve are constant, frenetic frills. Background voices are overly interpolating this main theme; *they're literally adding noise*, but doing so in a way that feels informed and constructive.

Not random notes (maximal energy), not perfect harmonies (curve-fitting low energy), but tantalizing chromatic deviations (overparameterized, but attracted). In the realms of language and music, I do not know if smoothly defined functions even exist—I do not dare suggest an integral be computed over Chopin's keys. Yes, we know musical notes and scales can be explained with very crisp mathematics, but any scale played alone and right, any sequence that is predictable to our ears, will be uninteresting. Music touches us when it build *tension*, when it creates a potential energy *to collapse back to* or increases kinetic energy by *playing around* those predictable, rational scales and notes.

The tension cannot be flippant or thoughtless. There is constantly in the mind of the chromatic composer an understanding of the tension being built, its boundedness, the pull of the consonant, lower energy state. At once imperfect but measured, Chopin's no. 11 in A minor sounds like an overly complex, but duly normalized concert. Perhaps I am clutching biased to these all-too-human *perfect imperfections* in the age of ever more aware artificial creators—god help us—but I believe that this extra noise, tamed to be regular, enriches the piece. We get to dance in and out of consonance and dissonance, feeling a momentum and an acceleration, not just a constant speed.

I think the imprecisions of language that cannot be computed as clearly as a syntax tree—the zeugmatic potential, the fun of puns, the inside jokes—all add to its power as a medium for shared experience. Yes, communication is about information, but we are not informing on events that occur in an objective world. Everything we see and know is warped in our lens; it must be transformed through the nested polynomials in our brains to even find a coherent semantic home. When we seek to sonically convey the conditions we find behind the filters of our mind, we cannot expect objective fidelity and we'd diminish the message if we had it.

The composition of a piece of art or comedic language are moves in Carse's *infinite game*---one that seeks not to arrive at a certain conclusion or goal, but to kick the conclusion down the road. Gardens are not

machines, for they seek to grow and enrich their surroundings with their presence, not to do their job so well that they disappear.

Music seeks not to disappear in consonance, giving no new information by being entirely predictable. Measured, bounded, guided tension is the fuel and fire of music. And apparently of overparameterized machine learning models.